

Basics of Docker

Diego Garrido Martín

*UST, University of Vic, Vic
Computational Biology of RNA Processing, CRG, Barcelona*



- Reproducibility in Bioinformatics
- What is Docker?
- Basic concepts and workflow
 - Docker CLI and Daemon
 - Images, Dockerfile and Docker registry
 - Containers
 - Volumes
- Hands-on

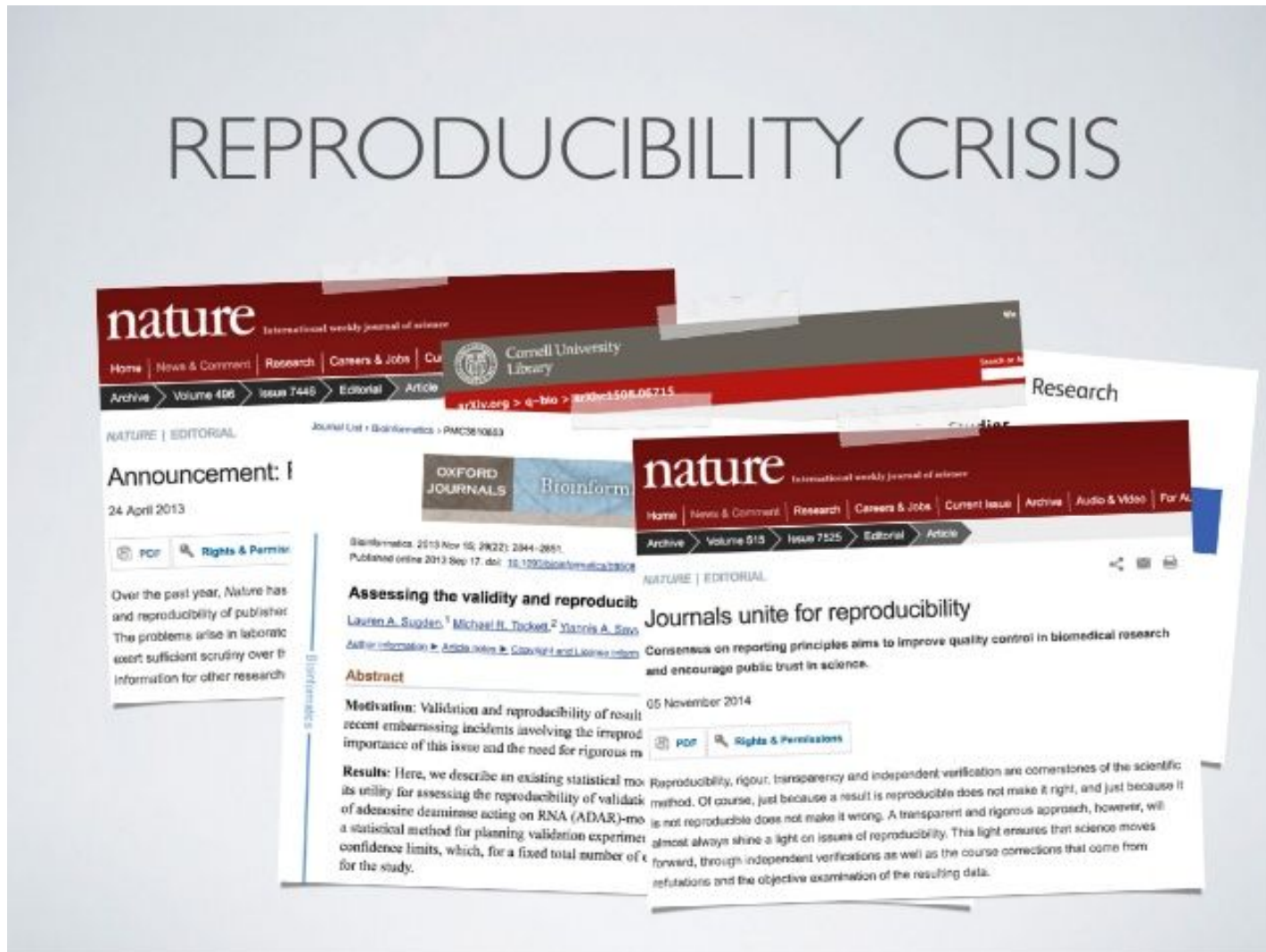
1. Reproducibility in Bioinformatics

Two major challenges in Bioinformatics:

1. Computation
2. **Reproducibility**

Reproducibility crisis

REPRODUCIBILITY CRISIS



Reproducibility in Bioinformatics

OPEN ACCESS Freely available online



Quantifying Reproducibility in Computational Biology: The Case of the Tuberculosis Drugome

Daniel Garijo¹, Sarah Kinnings², Li Xie³, Lei Xie⁴, Yinliang Zhang⁵, Philip E. Bourne^{3*}, Yolanda Gil^{6*}

¹ Ontology Engineering Group, Facultad de Informática, Universidad Politécnica de Madrid, Madrid, Spain, ² Department of Chemistry and Biochemistry, University of California San Diego, La Jolla, California, United States of America, ³ Skaggs School of Pharmacy and Pharmaceutical Sciences, University of California San Diego, La Jolla, California, United States of America, ⁴ Department of Computer Science, Hunter College, The City University of New York, New York, New York, United States of America, ⁵ School of Life Sciences, University of Science and Technology of China, Hefei, Anhui, China, ⁶ Information Sciences Institute and Department of Computer Science, University of Southern California, Los Angeles, California, United States of America

- Replicating the result of a typical computational biology paper requires ~ 280h



Think that reproducibility is not only important for others, but also for *the future yourself!*

Author's Local Machine

Mac OS 10.10.5
R 3.1.3
Hgu133plus2 v18
HeLa dataset
Source Code

223
Significant
Genes

Reader A

Ubuntu 14.04
R 3.2.4
Hgu133plus2 v19
HeLa dataset
Source Code

228
Significant
Genes

Reader B

Mac OS 10.10.4
R 3.3.0
Hgu133plus2 v20
HeLa dataset
Source Code

227
Significant
Genes

Complexity of bioinformatics workflows

- Many dependencies: binary tools, compilers, libraries, system tools, etc.
- Experimental nature of academic software: tends to be difficult to install and deploy.
- Heterogeneous executing platforms and system architecture: from your personal laptop to a supercomputer!

Solution?



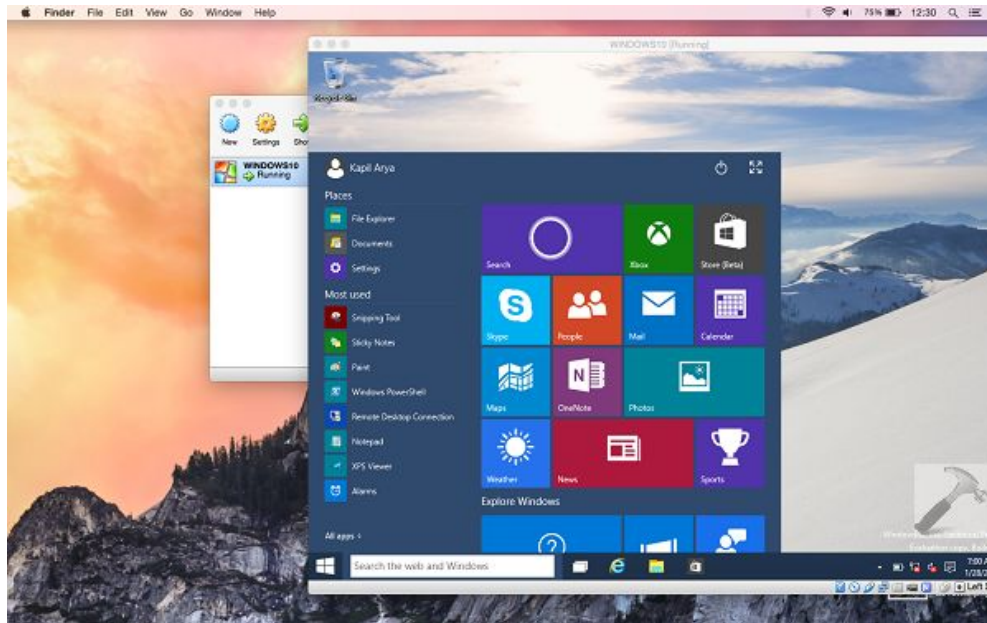
2. What is Docker

What is Docker?

- [Docker](#) is a software that performs OS-level *light* virtualization *aka* **containerization**.
- But what does this mean!? Essentially, containerization allows to package an application (i.e. a program) and its dependencies into an isolated, self-contained unit (*aka* **container**) that can be run on any Linux computer.
- Still difficult? Let's review then the concept of *virtual machine* (VM). A VM is essentially an **emulation** of a real computer (*full virtualization*). It allows running a **guest** OS (*virtual*) using your **host** (*physical, real-world*) hardware. In other words, the hardware is being virtualized to run multiple OS instances.
- Probably you have used a VM to run Ubuntu on Windows or Mac (or viceversa)

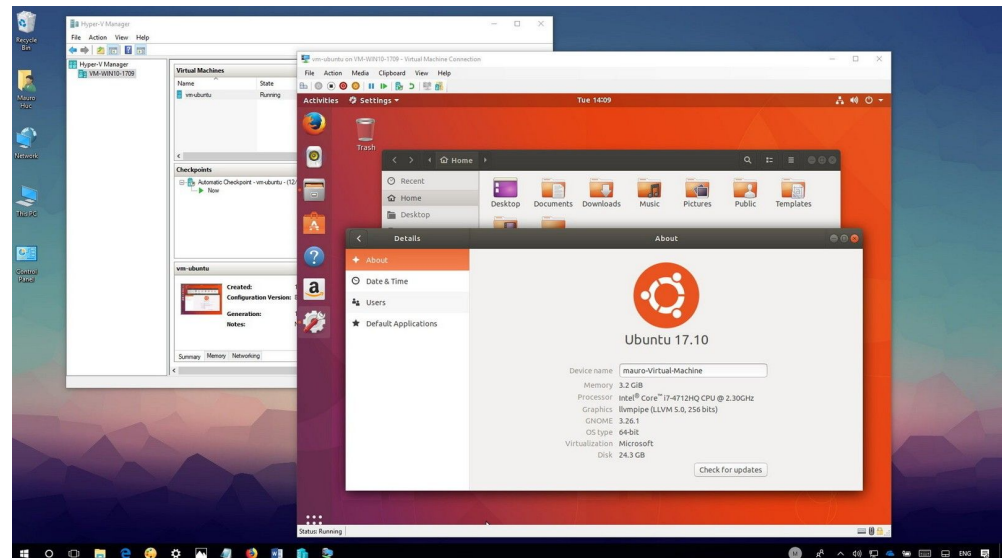


What is Docker?

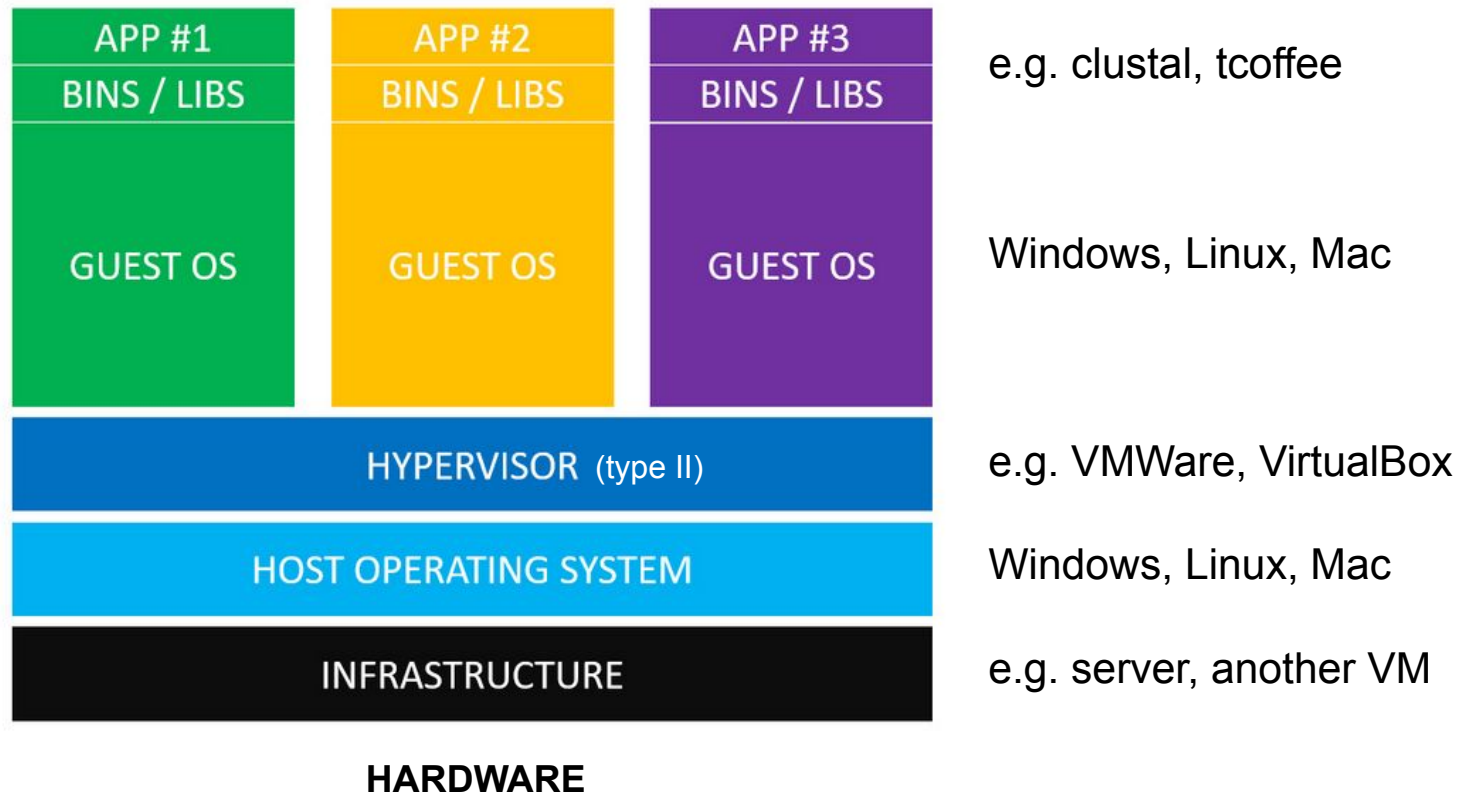


Using a virtual machine
running Windows 10 in
Mac ...

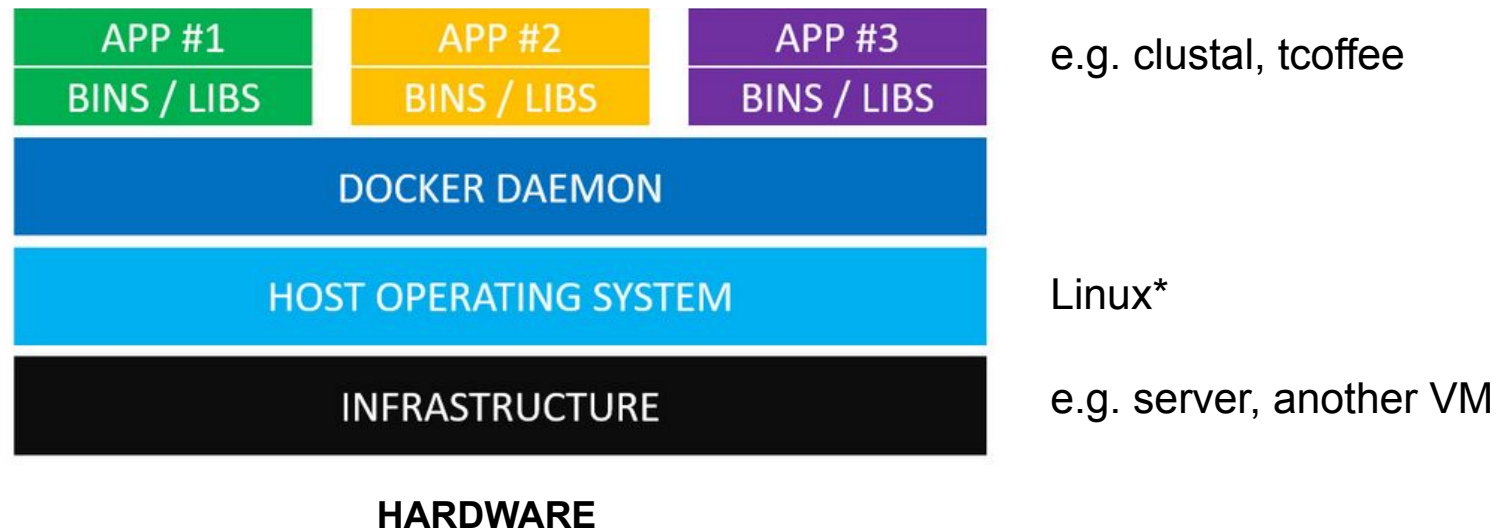
... and another one
running Ubuntu in
Windows 10.



Docker vs VMs

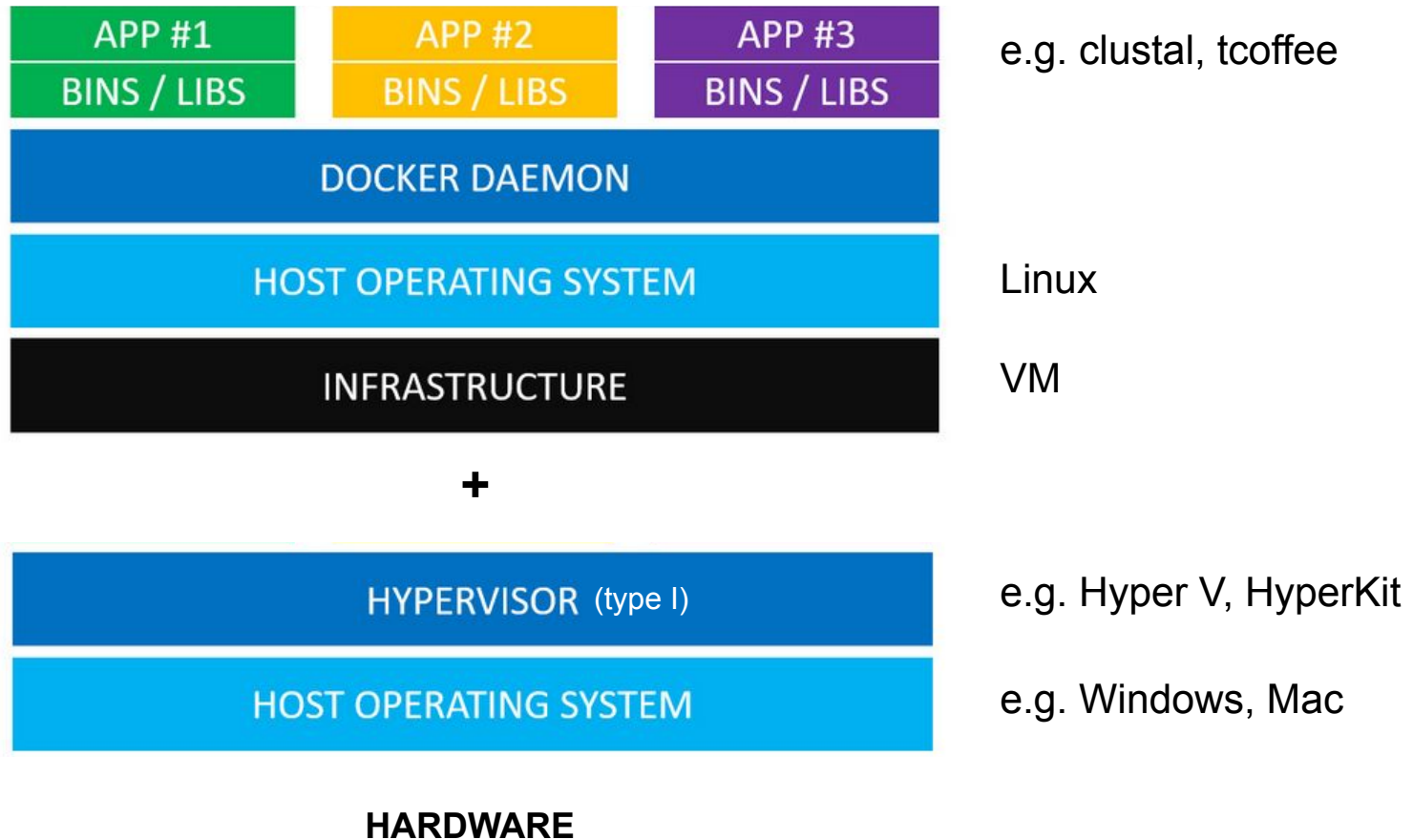


Docker vs VMs



* Docker can run natively also in recent versions of Windows and Mac, thanks to different virtualization solutions (type I hypervisors)

Docker vs VMs



Docker vs VMs

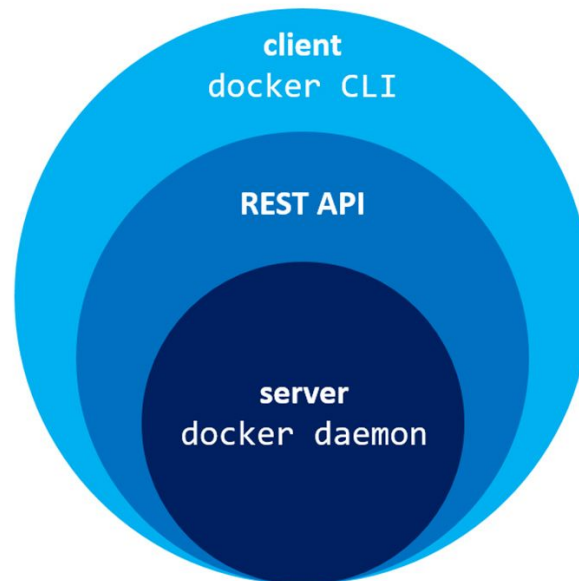
- Docker containers have many of the advantages of a VM, providing an isolated environment that allows running Linux applications in a reproducible manner.
- In contrast to VMs, containers share the OS and, where appropriate, bins/libraries with the host.
- Therefore, they are exceptionally *light*: Mb in size and take < 1 second to start vs Gb and minutes for a VM. Less resource usage (processor, RAM, etc.).
- Containers can be easily shared, enhancing collaboration
- Docker containers can run on a VM. Containers and VMs are not mutually exclusive and co-exist for many applications.

3. Basic concepts and Docker workflow

The Docker Engine

The *Docker Engine* is a client-server application with these major components:

- A server which is a type of long-running program called a daemon process (the `dockerd` command).
- A REST API which specifies interfaces that programs can use to talk to the daemon and instruct it what to do.
- A command line interface (CLI) client (the `docker` command).

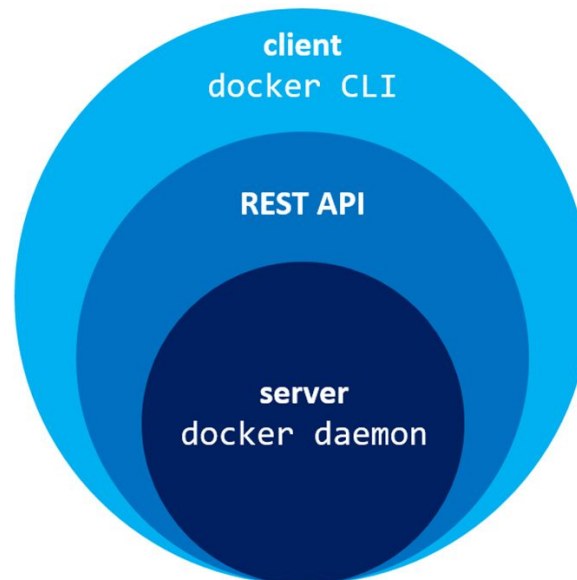


`docker --help`

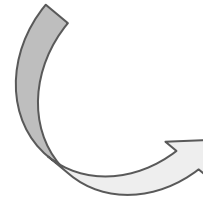
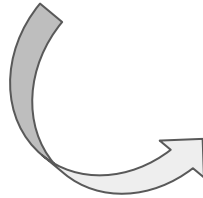
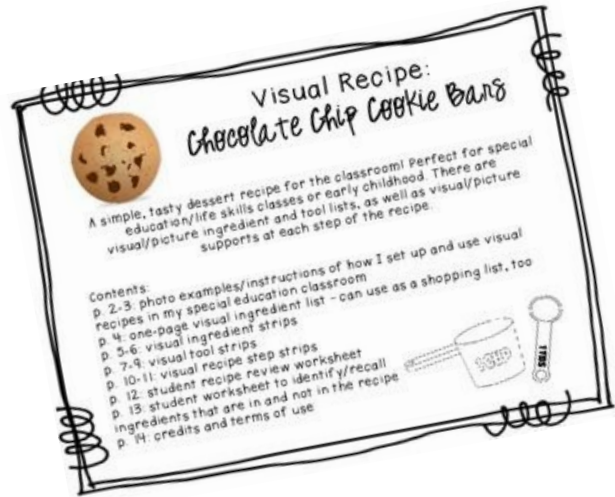
The Docker Engine

The CLI uses the Docker REST API to **control or interact** with the Docker daemon through scripting or direct CLI **commands**. Many other Docker applications use the underlying API and CLI. The CLI is the primary way through which many Docker users interact with Docker.

The daemon creates and manages Docker **objects**, such as images, containers, networks, and volumes.



Docker objects: images



Docker objects: images

- An ***image*** is a read-only template with instructions for creating a Docker container. Often, an image is based on another one, with some additional customization.
- You can create your own images or use those created by others that are published in a **registry**.
- To build your own image, you write a ***Dockerfile*** with a simple syntax defining the steps needed to create it. Then use **docker build**. Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt.

Docker objects: images

Dockerfile

```
FROM debian:stable

LABEL maintainer "you <your_email>" \
      version "0.1" \
      description "your_description"

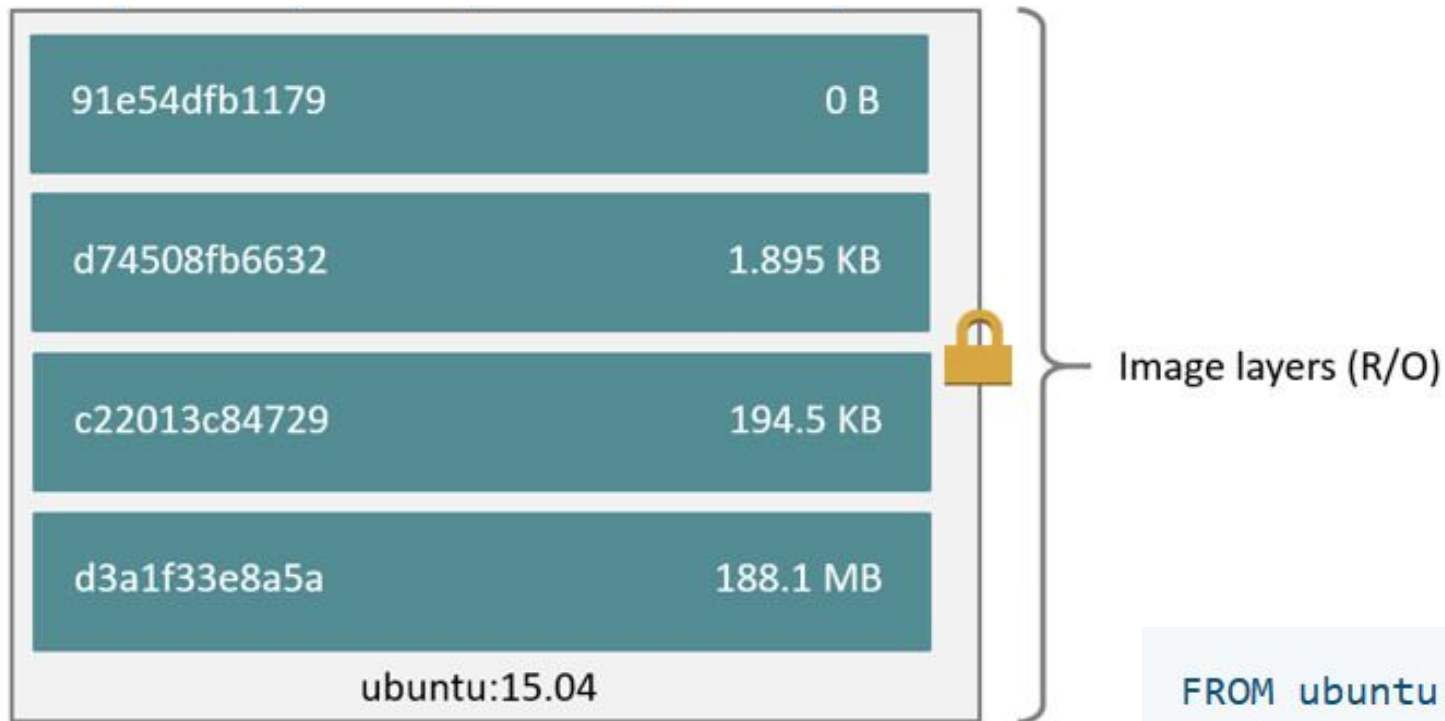
# Install R
RUN apt-get update && apt-get install -y r-base

# Install R package 'optparse'
RUN R -e 'install.packages("optparse", repos = "http://cloud.r-project.org/)"

RUN mkdir /scripts

ADD scripts/script.R /scripts
```

Docker objects: images



(based on ubuntu:15.04 image)

```
FROM ubuntu:15.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

Docker objects: images

`docker images`

List images on this machine

`docker build --tag <name:tag> .`

Create image *name* using this folder's Dockerfile
Adding a *tag* is optional, default *latest*

`docker rmi <image id>`

Remove specified image from this machine

`docker rmi $(docker images -q)`

Remove all images from this machine



`docker <command> --help`

Docker registry

The **registry** is a server side application that lets you **store and distribute** Docker images. [Docker Hub](#) is a cloud-based registry service, where you can have:

- [Repositories](#), which allow you to push images from a local Docker daemon.
- [Automated Builds](#), which link to a source code repository (e.g. GitHub) and trigger an image rebuild process on Docker Hub when changes are detected.

They can be public or private. You can **search** Docker Hub and **pull** images without a Docker account. However, to **push** images, leave comments, or to *star* a repository, **you need to log in using a [Docker ID](#)**.

Docker registry

Log in using your Docker credentials

```
docker login
```

Search for <image> in DockerHub

```
docker search <image>
```

Name and tag <image> for upload to DockerHub

```
docker tag <image> username/repository:tag
```

Upload image to DockerHub

```
docker push username/repository:tag
```

Download image from DockerHub

```
docker pull username/repository:tag
```

Run image from DockerHub if not present locally

```
docker run username/repository:tag
```

Docker objects: containers

- A **container** is a running instance of an image: what the image becomes in memory when executed (i.e. a user process).
- A container is isolated from other containers and the host machine.
- A container is defined by its image as well as any configuration options you provide to it when you create or start it. You can run a container from an image using **docker run**.

```
docker ps -a
```

```
docker rm <id>
```

```
docker rm $(docker ps -a -q)
```

```
docker run <image> <command>
```

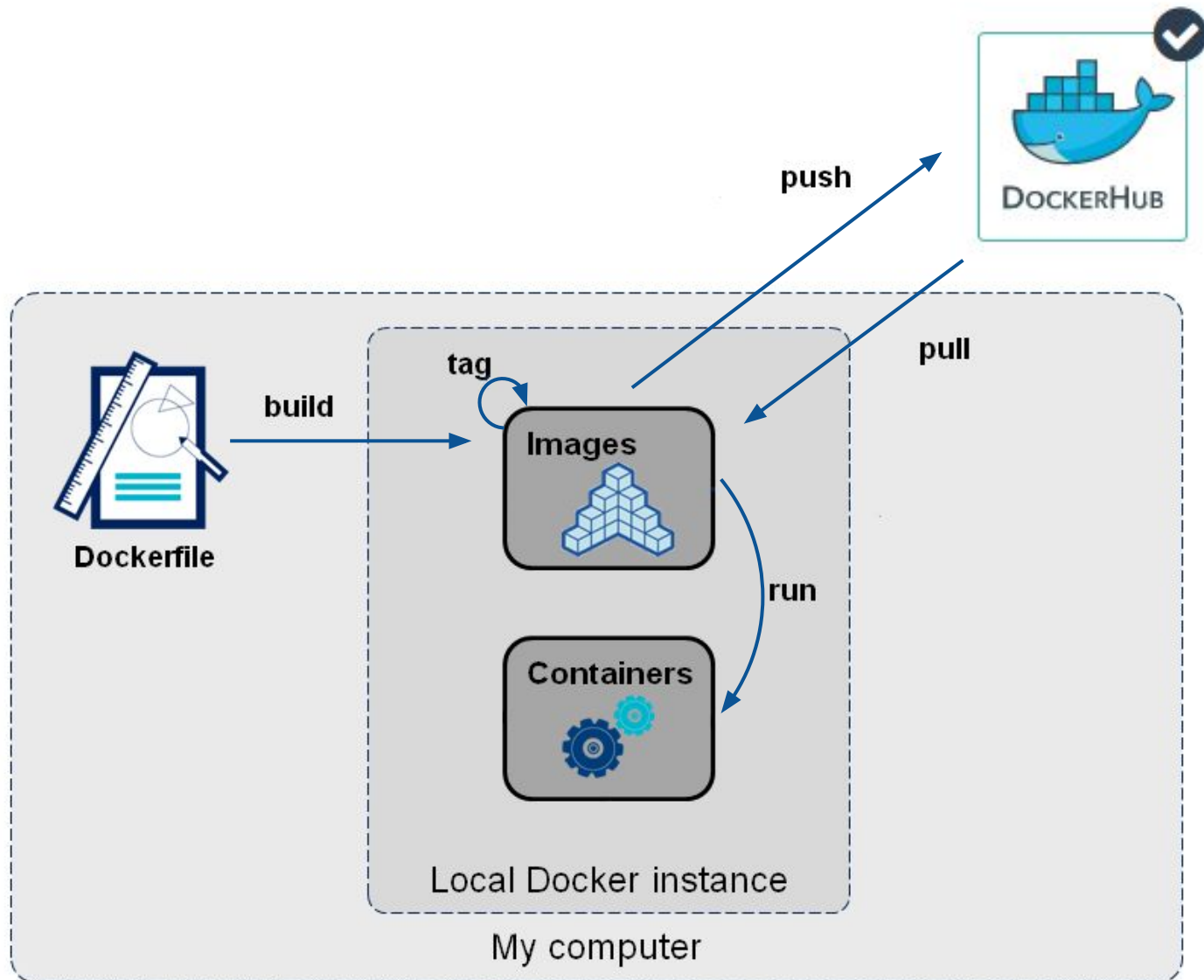
```
# List all containers, even if not running
```

```
# Remove container
```

```
# Remove all containers
```

```
# Run a command in a container
```

Docker basic workflow



Docker objects: containers

Example: The following command runs the *ubuntu* container from the *ubuntu* image, attaches interactively to your local command-line session, and runs **ls**.

```
docker run -i -t ubuntu ls
```

1. If you do not have the *ubuntu* image locally, Docker pulls it from your configured registry (as if you had run **docker pull** manually).
2. Docker creates a new container (as if you had run **docker create** manually).
3. Docker allocates a read-write filesystem to the container, as its final layer. This allows a running container to create or modify files and directories in its local filesystem.

Docker objects: containers

4. Docker creates a network interface to connect the container to the default network, since you did not specify any networking options. This includes assigning an IP address to the container. By default, containers can connect to external networks using the host machine's network connection.
5. Docker starts the container and executes `ls`. Because the container is running interactively (`-i`) and attached to your terminal (`-t`), you can provide input using your keyboard while the output is printed to your terminal. Within the container, you have *root* permissions.
6. When you type `exit`, the container **stops but is not removed**. You can start it again or remove it.

Docker objects: containers

`docker run [options]`

`-i, --interactive`

Keep STDIN open even if not attached

`-t, --tty`

Allocate a pseudo-TTY (attach to terminal)

`-m, --memory <bytes>`

Memory limit

`-w, --workdir <string>`

Working directory inside the container

`--rm`

Automatically remove the container when it exits

`--name <string>`

Assign a name to the container

`-v, --volume list`

Bind mount a volume



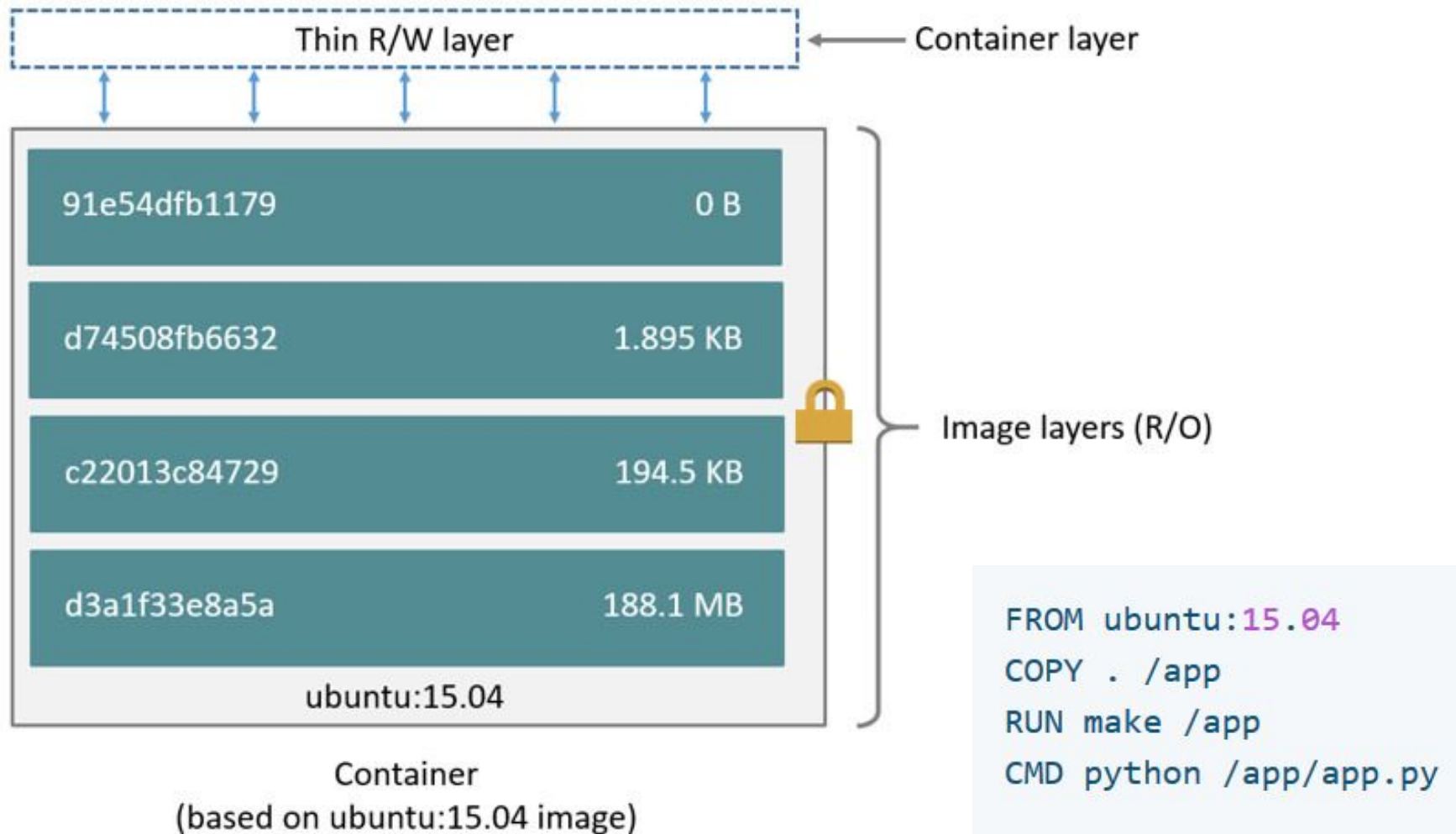
`docker run --help`

Docker objects: containers

Your Dockerfile:

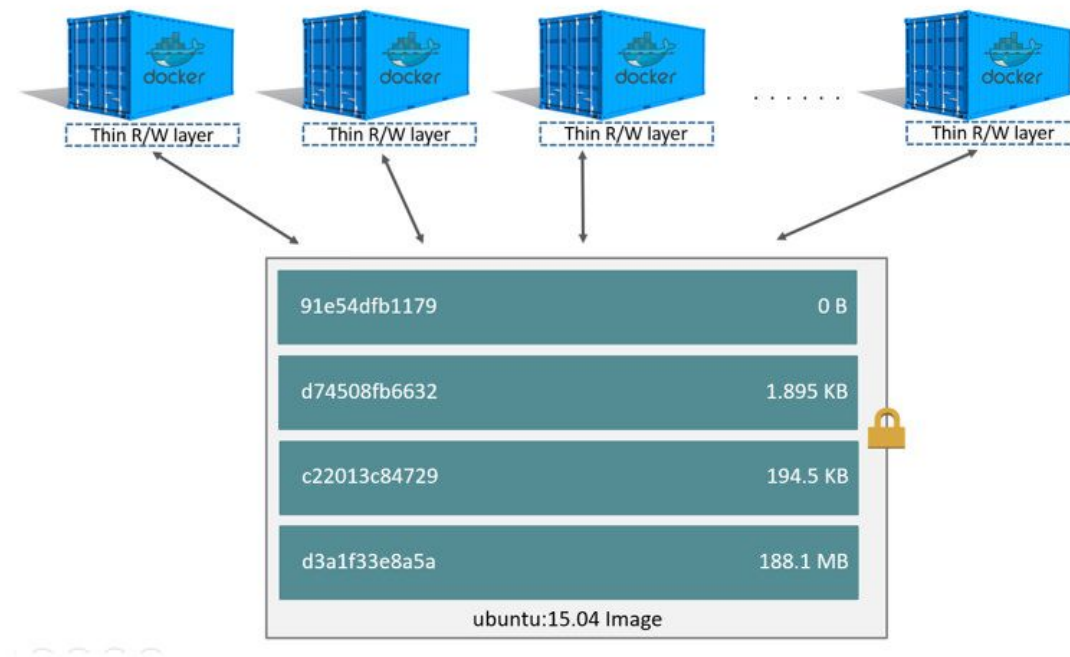
```
FROM ubuntu:15.04  
COPY . /app  
RUN make /app  
CMD python /app/app.py
```

Docker objects: containers



Docker objects: containers

- The major difference between a container and an image is the top writable layer. All writes to the container that add new or modify existing data are stored in this writable layer. When the container is deleted, the writable layer is also deleted. The underlying image remains unchanged.



- Because each container has its own writable container layer, and all changes are stored in this container layer, multiple containers can share access to the same underlying image and yet have their own data state.

Docker objects: volumes

- **Storage drivers** allow you to create data in the writable layer of your container. However, **the files won't be persisted after the container is no longer running, and both read and write speeds are low.**
- Docker has two main options for containers to store files in the host machine, so that the files are persisted even after the container stops: ***volumes*** and ***bind mounts***. These strategies also improve performance.

Docker objects: volumes

A **volume** or logical drive is a single accessible storage area with a single file system.

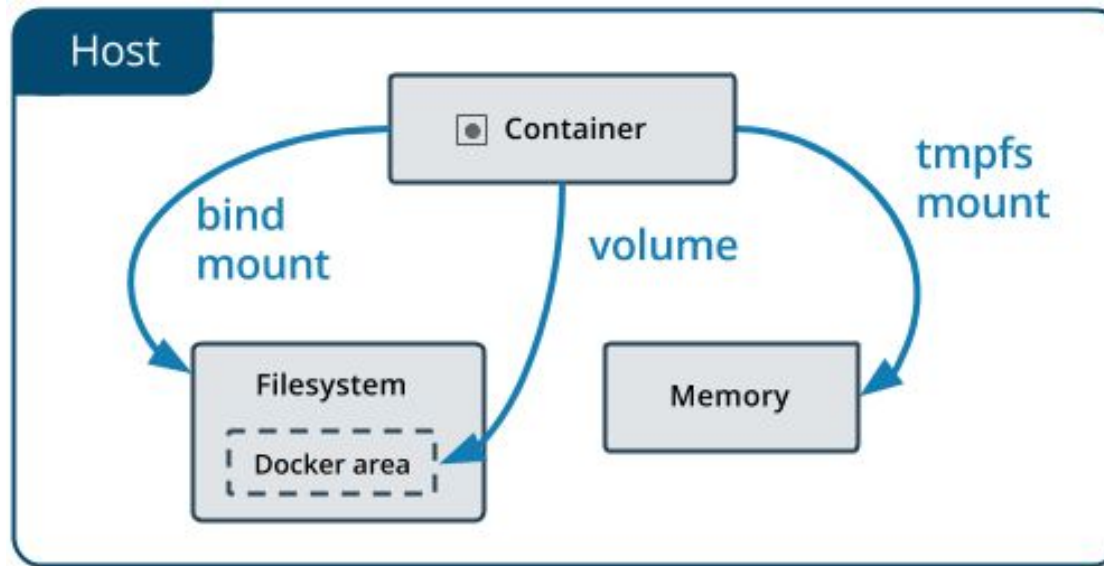
example in
Windows

Physical disk	Partition	Filesystem	Drive letter
Hard Disk 1	Partition 1	NTFS	C:
	Partition 2	FAT32	D:
Hard Disk 2	Partition 1	FAT32	E:

- "C:", "D:", and "E:" are volumes.
- Hard Disk 1 and Hard Disk 2 are physical disks.
- Any of these can be called a "drive".

Mounting is a process by which the operating system makes files and directories on a storage device (e.g. hard drive) available for users to access via the computer's file system.

Docker objects: volumes



- **Volumes** are stored in a part of the host filesystem *managed by Docker* (`/var/lib/docker/volumes/` on Linux). Non-Docker processes should not modify them.
- **Bind mounts** may be stored *anywhere* on the host. Non-Docker processes on the host or a container can modify them.
- **tmpfs mounts** are stored in the host's memory, and are never written to the host's filesystem.

Docker objects: volumes

- Volumes

- **Created and managed by Docker**, isolated from the host's functionality. Stored within a directory on the host. **This directory is what is mounted into the container.**
- You can create them explicitly (`docker volume create`) or implicitly when creating/running a container.
- Can be named or anonymous, are managed by Docker CLI commands and can be mounted into multiple containers simultaneously. When unused they are not automatically removed.



`docker volume --help`

- Bind mounts

- **A file or directory on the *host machine* is mounted into a container**
- Referenced by its full path on the host machine, created on demand if it does not yet exist.
- You can't use Docker CLI commands to directly manage bind mounts.
- Allow changes in the host filesystem via processes running in a container, **including creating, modifying, or deleting important system files or directories.**

Docker objects: volumes

- Volumes

```
docker run --mount \
    source=<volume>,target=<target> <container>
```

```
docker run -v <volume>:<target> <container>
```

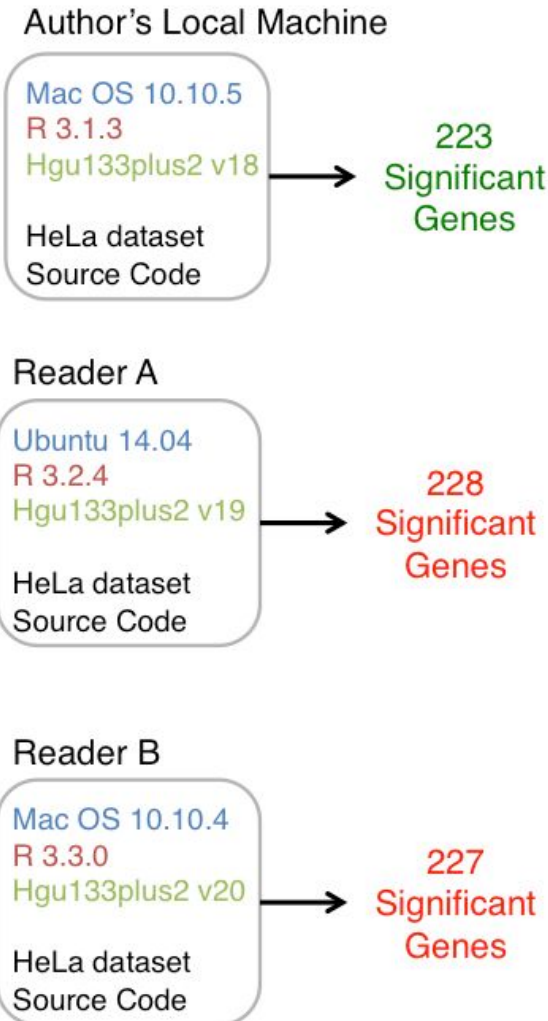
- Bind mounts

```
docker run --mount \
    type=bind,source=<path>,target=<target> <container>
```

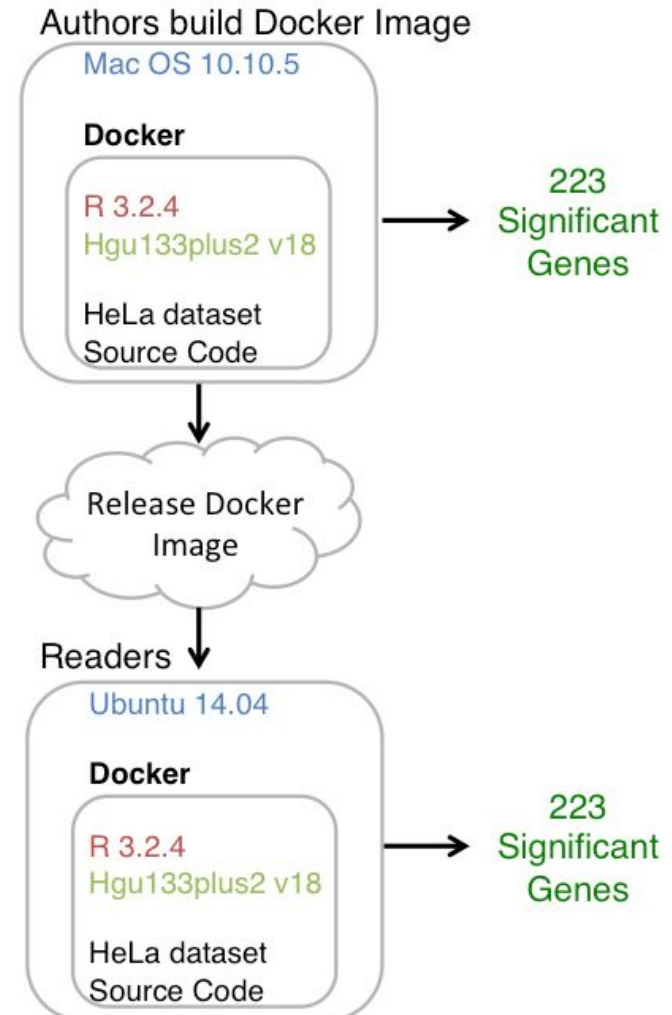
```
docker run -v <path>:<target> <container>
```

Reproducibility in Bioinformatics with Docker

A. CURRENT SYSTEM



B. CONTAINER-BASED APPROACH



Hands On

https://github.com/dgarrimar/teaching_msc/wiki/Docker-hands-on

Additional resources

- Docker documentation
 - <https://docs.docker.com>